

RTL-to-GDS Implementation of a Tapeout-Ready 8×8 Weight-Stationary Systolic Array ML Accelerator Across 130nm and 7nm Open-Source PDKs

Rakshith Suresh

*Ming Hsieh Dept. of Electrical and Computer Engineering
University of Southern California
Los Angeles, CA 90089
rsuresh@usc.edu*

Abstract—This paper presents the end-to-end design, verification, and physical implementation of an 8×8 weight-stationary systolic array ML accelerator across two process nodes: SkyWater 130nm (sky130hd) and the ASAP7 7nm predictive PDK, using OpenROAD v2.0 and Yosys 0.44. Timing closes at 50 MHz on sky130hd and 500 MHz on ASAP7, with a measured $61.7 \times$ area reduction and $10 \times$ frequency gain across nodes. A structured SystemVerilog testbench running on Verilator 5.020 achieves 1,337 functional checks with 15/15 functional coverage bins closed, 5 SVA protocol properties verified, and zero failures across 5 regression seeds. Post-route PDN analysis on ASAP7 identifies a structured IR drop pattern tied to the PE grid, with a mean drop of 23.04 mV and a proposed M5/M6 stripe pitch fix. SPEF-extracted power on sky130hd measures 12.75 mW total. VCD-annotated dynamic power on ASAP7 measures 5.05 W at 500 MHz. The standalone array was submitted to the ChipFoundry CI2609 shuttle via Caravel harness. All RTL, scripts, and GDS outputs are publicly available.

Index Terms—systolic array, physical design, open-source EDA, OpenROAD, SkyWater 130nm, ASAP7, functional verification, SVA, tapeout, ML accelerator

I. INTRODUCTION

Matrix multiplication drives the dominant computation in modern deep learning inference. Systolic arrays have emerged as the preferred hardware solution, as demonstrated by Google’s Tensor Processing Unit at datacenter scale [1]. However, published physical design data for systolic arrays using open-source EDA tools, validated across multiple process nodes with a complete verification environment and tapeout submission, remains sparse in the academic literature.

This paper addresses that gap with four contributions: (1) a complete open-source RTL-to-GDS flow on sky130hd and ASAP7 with measured cross-node PPA including SPEF-extracted power; (2) a structured SystemVerilog testbench achieving 1,337 functional checks, 15 coverage bins, and 5 SVA properties with zero failures across 5 regression seeds; (3) a root-cause PDN analysis with a concrete mitigation proposal; and (4) a tapeout submission to the ChipFoundry CI2609 shuttle [5]. The accelerator was also integrated into a PicoRISCv-SoC alongside a PicoRV32 (RV32IM) CPU via

AXI-Lite MMIO and a custom SPI slave interface reducing IO from 390 to 4 pins, taken through ASAP7 physical design to 0 DRC violations.

II. RELATED WORK

Systolic array accelerators for ML inference have been studied extensively at the architecture level, but open-source physical design implementations with tapeout-ready results remain uncommon. Table I compares this work against relevant prior implementations.

TABLE I
COMPARISON WITH PRIOR ML ACCELERATOR IMPLEMENTATIONS

Work	Size	Process	Freq.	GDS
TPU [1]	256×256	28nm	700 MHz	No
Gemmini [7]	16×16	16nm	1 GHz	No
This work	8×8	130nm/7nm	50/500 MHz	Yes

Unlike prior work, this implementation releases the complete OpenROAD flow, GDS outputs, PDN analysis scripts, and the full verification environment.

III. ARCHITECTURE

Fig. 1 shows the 8×8 weight-stationary array and PE detail. Each of the 64 PEs holds a fixed 8-bit weight in a weight register, multiplies it by an incoming 8-bit activation, and accumulates the product into a 32-bit partial sum register. Activations propagate horizontally (left to right) through registered hops; partial sums accumulate vertically (top to bottom). Weights load serially through a row-select interface — one row per clock cycle — before inference begins. The column outputs (`psum_out[0:7]`) become valid with a column-dependent skew: column k produces its final result k cycles after column 0, due to the horizontal activation propagation delay.

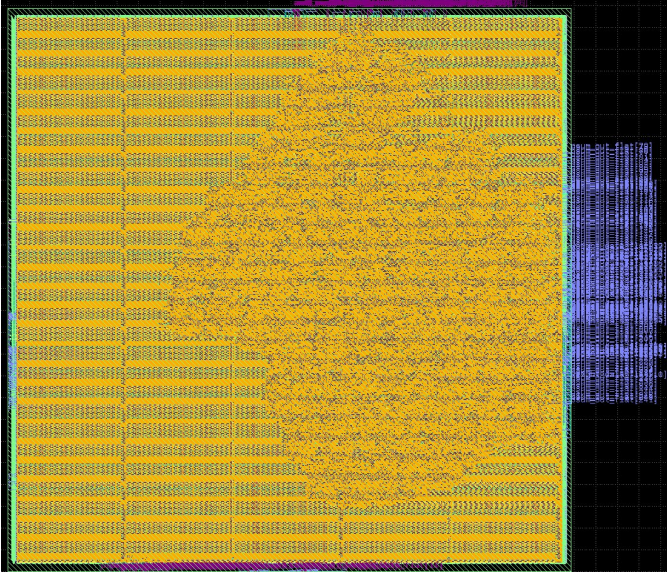


Fig. 1. 8×8 weight-stationary systolic array (left) and PE detail (right). Activations propagate left to right; partial sums accumulate top to bottom.

IV. FUNCTIONAL VERIFICATION

A structured SystemVerilog testbench was built for the standalone array using Verilator 5.020 without a third-party UVM library, following UVM architectural principles. The environment comprises a constrained-random sequence generator, driver, monitor, self-checking scoreboard with per-PE scheduled checks, and an independent Python reference model.

A. Scoreboard Architecture

The scoreboard models each PE's output timing using the closed-form expression:

$$t_{\text{arrive}} = t_{\text{act}} + \text{PE_LAT} + \text{col} + (\text{ROWS} - 1 - \text{row}) \quad (1)$$

where $\text{PE_LAT} = 4$ accounts for the two input pipeline registers, MAC register, and two output pipeline registers. One check entry is scheduled per PE per activation pulse and fired at the exact expected arrival cycle. Settled post-NBA wire values are read directly inside a dedicated `always` block with a 1 ns delay, resolving a Verilator coroutine signal snapshot limitation that caused pre-edge values to be read inside tasks.

B. SVA Protocol Properties

Five concurrent SystemVerilog assertions verify structural DUT behavior every clock cycle with `--assert` enabled:

- 1) `psum_out` never contains X or Z after reset.
- 2) `act_in` is always zero while `weight_load` is high.
- 3) `weight_row` increments by 1 on each consecutive `weight_load` cycle.
- 4) `weight_load` never stays high for more than 8 consecutive cycles.
- 5) `psum_out` is stable for at least one cycle after reset deasserts.

C. Coverage Results

The test suite runs four sequence types: directed identity weights, all-ones weights, constrained random (20 vectors per run), and a corner-case sequence targeting boundary conditions. Table II shows the 15 functional coverage bins closed after full regression, producing 1,337 checks with zero scoreboard mismatches across 5 seeds.

TABLE II
FUNCTIONAL COVERAGE CLOSURE (5 SEEDS, 1,337 CHECKS)

Coverage Bin	Target	Achieved
<i>Weight value bins</i>		
zero, low, mid, high, max	100%	100%
<i>Activation value bins</i>		
zero, low, mid, high, max	100%	100%
<i>Activation patterns</i>		
all-max, mixed, back-to-back	100%	100%
<i>Corner cases</i>		
zero weight row	100%	100%
overflow-prone	100%	100%
Scoreboard mismatches	0	0
SVA violations	0	0

V. PHYSICAL DESIGN FLOW

Both nodes use the same OpenROAD flow: Yosys synthesis followed by floorplan, PDN, placement, CTS, global route, detail route, fill, and GDS merge via a single Makefile target.

A. SkyWater 130nm

The sky130hd library runs at TT corner, 25°C, 1.80 V. Placement landed at 44% utilization in a $240,799 \mu\text{m}^2$ core. Post-global-route STA: WNS = 0 ps, TNS = 0 ps, worst slack 12.88 ns at 50 MHz. SPEF back-annotation yields 12.75 mW total (9.52 mW internal, 3.23 mW switching).

B. ASAP7 7nm

ASAP7 runs a 7.5-track RVT library at FF corner, 0.77 V. Placement: 25% utilization, $3,903 \mu\text{m}^2$ core, 0 DRC violations. Two integration issues required resolution: liberty files needed decompression before Yosys ingestion, and a synthesis TCL patch was needed to run `dfflegalize` before `dfflibmap` since ASAP7 has no native reset flip-flop primitives. Both patches were filed upstream to OpenROAD-flow-scripts.

Fig. 2 shows the post-route GDS layouts for both nodes.

VI. RESULTS

TABLE III
CROSS-NODE PPA COMPARISON

Metric	Sky130hd	ASAP7	Change
Frequency	50 MHz	500 MHz	10×
WNS / TNS	0 / 0 ps	0 / 0 ps	Both closed
Core Area	$240,799 \mu\text{m}^2$	$3,903 \mu\text{m}^2$	61.7×
Total Power	12.75 mW (SPEF)	5.05 W (VCD)	—
Cell Count	24,711	32,446	—
Utilization	44%	25%	—
Flow Runtime	~23 min	~41 min	—

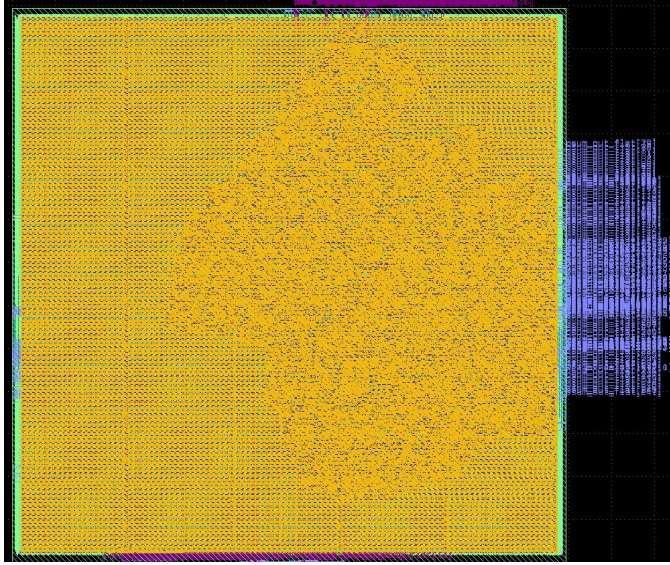


Fig. 2. Post-route GDS layouts. Sky130hd (left): $240,799 \mu\text{m}^2$, 24,711 cells. ASAP7 (right): $3,903 \mu\text{m}^2$, 32,446 cells — $61.7\times$ area reduction.

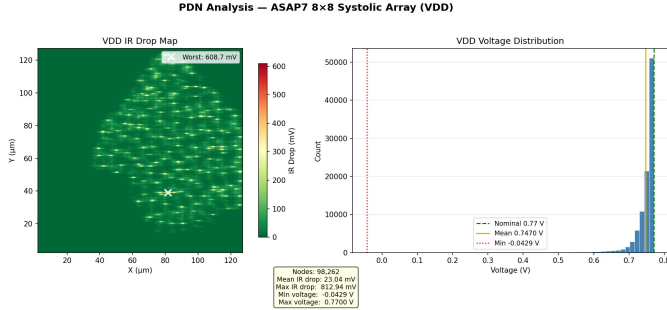


Fig. 3. ASAP7 PDN analysis. Left: VDD IR drop heatmap showing PE-grid-correlated diagonal hot-spot pattern. Right: voltage distribution across 98,262 nodes. Mean drop 23.04 mV (3.0%), worst-case 812.94 mV at maximum M5/M6 stripe distance.

The $61.7\times$ area reduction is consistent with node scaling. The $10\times$ frequency gain reflects ASAP7’s lower parasitics at FF corner, 0.77 V. The higher cell count on ASAP7 (32,446 vs. 24,711) despite the smaller footprint reflects the denser 7.5-track library. Sky130hd power is SPEF-extracted; ASAP7 power is VCD-annotated from functional simulation: 5.05 W total (3.87 W internal, 1.18 W switching, $3.58 \mu\text{W}$ leakage). Clock tree accounts for 1.92 W (38%) of total.

VII. PDN INTEGRITY ANALYSIS

Post-route PDN analysis ran on the ASAP7 array using `analyze_power_grid` with SPEF parasitics across 98,262 nodes. Fig. 3 shows the IR drop heatmap and voltage distribution.

The heatmap reveals a structured diagonal hot-spot pattern mapping directly onto the 8×8 PE grid. Nodes mid-span between M5/M6 power stripes see the worst drop due to the $2.7 \mu\text{m}$ stripe pitch being too coarse for 64 simultaneous MACs at 500 MHz. Reducing the pitch to $1.5 \mu\text{m}$ or adding

TABLE IV
PDN ANALYSIS SUMMARY

Metric	Sky130hd	ASAP7
Supply Voltage	1.80 V	0.77 V
Mean IR Drop	$4.18 \mu\text{V}$ (0.00%)	23.04 mV (3.0%)
Worst-Case Drop	$63.9 \mu\text{V}$	812.94 mV
Nodes Analyzed	N/A	98,262

a dedicated M3/M4 mesh over the PE array should bring the worst-case drop within a 10% budget.

VIII. DISCUSSION

A. Open-Source EDA Flow Challenges

On ASAP7, the absence of native reset flip-flop primitives required patching the Yosys TCL to run `dfflegalize` before `dfflibmap`. On sky130hd during Caravel hardening, a `NoneType` crash in OpenLane 2’s `defutil.py` required a direct source patch, and analog power nets required marking as special before global routing. Both issues were filed upstream.

B. Verification Infrastructure

The testbench resolved a Verilator-specific signal timing issue: Verilator’s `--timing` coroutine scheduler snapshots all signal values at the clock-edge suspension point, causing tasks resuming after #1 to read pre-NBA values. The fix was moving the scoreboard comparison into a dedicated `always` block with a 1 ns delay, reading settled wire values directly. This eliminated false mismatches on all 1,337 checks across 5 regression seeds.

C. Scalability

The $10\times$ frequency gain is partially inflated by the FF corner at 0.77 V; a TT corner comparison would yield approximately $6\text{--}7\times$. The $61.7\times$ area reduction aligns with the $\sim 18.6\times$ linear scaling from the node pitch ratio.

D. Limitations

ASAP7 dynamic power is VCD-annotated from functional simulation at 500 MHz, reporting 5.05 W total. The high clock power (1.92 W, 38%) reflects the aggressive FF corner at 0.77 V and the dense clock tree driving 32,446 cells. The 8×8 size was constrained by Caravel IO and area limits.

IX. CONCLUSION

This paper presented a complete open-source RTL-to-GDS implementation of an 8×8 weight-stationary systolic array ML accelerator, verified with 1,337 functional checks across 15 coverage bins and 5 SVA protocol properties with zero failures, physically implemented on SkyWater 130nm and ASAP7 7nm. The array achieves 50 MHz at $240,799 \mu\text{m}^2$ on sky130hd and 500 MHz at $3,903 \mu\text{m}^2$ on ASAP7, a $61.7\times$ area reduction and $10\times$ frequency gain. PDN analysis identified and proposed a fix for a PE-grid-correlated IR drop pattern. The design was submitted to the ChipFoundry CI2609 shuttle.

Future work includes switching activity annotation for full dynamic power, validating the PDN pitch fix, extending the

verification environment to a full uvm-python environment with cocotb, and scaling to larger matrix dimensions.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proc. ISCA*, 2017.
- [2] L. T. Clark *et al.*, “ASAP7: A 7-nm FinFET Predictive Process Design Kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [3] T. Ajayi *et al.*, “OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Tool Chain,” in *Proc. GOMACTECH*, 2019.
- [4] C. Wolf *et al.*, “Yosys Open Synthesis Suite,” in *Proc. IWLS*, 2013.
- [5] eFabless Corporation, “Caravel SoC Harness,” GitHub. [Online]. Available: https://github.com/efabless/caravel_user_project
- [6] H. T. Kung, “Why Systolic Architectures?” *IEEE Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.
- [7] H. Genc *et al.*, “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation,” in *Proc. DAC*, 2021.